

同济大学计算机系

计算机组成原理实验报告



学 号 2452769

姓 名 幸可函

专 业 计算机科学与技术

授课老师 张冬冬

一、实验内容

使用 Verilog 实现 32 位无符号除法器 and 32 位有符号除法器，完成模块编写与仿真测试。无符号除法器对两个 32 位无符号数进行除法运算，输出 32 位商 q 和 32 位余数 r ；有符号除法器对两个 32 位有符号数进行除法运算，输出 32 位商 q 和 32 位余数 r ，且余数符号与被除数符号相同。除法器工作过程中设置忙信号 $busy$ 表示运算状态，除数为 0 时结果未知。

二、硬件逻辑图

略。

三、模块建模

(1) 无符号除法器

```
module DIVU(  
    input [31:0] dividend,  
    input [31:0] divisor,  
    input start,  
    input clock,  
    input reset,  
    output [31:0] q,  
    output [31:0] r,  
    output reg busy  
);  
    reg [4:0] count;  
    reg [31:0] reg_q;  
    reg [31:0] reg_r;  
    reg [31:0] reg_b;  
    reg r_sign;  
  
    wire [32:0] sub_add = r_sign ? ({reg_r,reg_q[31]} +  
    {1'b0,reg_b}):({reg_r,reg_q[31]} - {1'b0,reg_b});  
  
    assign q = reg_q;  
    assign r = r_sign ? (reg_r + reg_b) : reg_r;  
  
    always @(posedge clock or posedge reset) begin  
        if (reset == 1) begin  
            count <= 5'b0;  
            reg_q <= 32'b0;  
            reg_r <= 32'b0;  
            reg_b <= 32'b0;
```

```

        r_sign <= 1'b0;
        busy <= 1'b0;
    end else begin
        if (start) begin
            reg_r <= 32'b0;
            r_sign <= 0;
            reg_q <= dividend;
            reg_b <= divisor;
            count <= 5'b0;
            busy <= 1'b1;
        end else if (busy) begin
            reg_r <= sub_add[31:0];
            r_sign <= sub_add[32];
            reg_q <= {reg_q[30:0],~sub_add[32]};
            count <= count + 5'b1;
            if (count == 5'd31) busy <= 0;
        end
    end
end
endmodule

```

无符号除法器 DIVU 用于实现 32 位无符号数除法运算，输入被除数 dividend、除数 divisor，在 start 有效后开始运算，输出 32 位商 q、32 位余数 r，并用 busy 表示当前处于运算状态。模块内部采用部分余数、商寄存器和除数寄存器配合计数器完成逐次迭代运算，运算结束后得到最终商和余数。

(2) 有符号除法器

```

module DIV(
    input [31:0] dividend,
    input [31:0] divisor,
    input start,
    input clock,
    input reset,
    output [31:0] q,
    output [31:0] r,
    output reg busy
);

    reg [4:0] count;
    reg [31:0] reg_q;
    reg [31:0] reg_r;
    reg [31:0] reg_b;
    reg core_r_sign;

```

```

reg q_sign;
reg r_sign_out;

wire [31:0] dividend_abs;
wire [31:0] divisor_abs;
wire [32:0] sub_add;
wire [31:0] r_unsigned;

assign dividend_abs = dividend[31] ? (~dividend + 32'b1) : dividend;
assign divisor_abs = divisor[31] ? (~divisor + 32'b1) : divisor;

assign sub_add = core_r_sign ? ({reg_r, reg_q[31]} + {1'b0, reg_b}) :
                              ({reg_r, reg_q[31]} - {1'b0, reg_b});

assign r_unsigned = core_r_sign ? (reg_r + reg_b) : reg_r;

assign q = q_sign ? (~reg_q + 32'b1) : reg_q;
assign r = r_sign_out ? (~r_unsigned + 32'b1) : r_unsigned;

always @(posedge clock or posedge reset) begin
    if (reset == 1'b1) begin
        count <= 5'b0;
        reg_q <= 32'b0;
        reg_r <= 32'b0;
        reg_b <= 32'b0;
        core_r_sign <= 1'b0;
        q_sign <= 1'b0;
        r_sign_out <= 1'b0;
        busy <= 1'b0;
    end
    else begin
        if (start) begin
            reg_r <= 32'b0;
            core_r_sign <= 1'b0;
            reg_q <= dividend_abs;
            reg_b <= divisor_abs;
            q_sign <= dividend[31] ^ divisor[31];
            r_sign_out <= dividend[31];
            count <= 5'b0;
            busy <= 1'b1;
        end
        else if (busy) begin
            reg_r <= sub_add[31:0];
            core_r_sign <= sub_add[32];
        end
    end
end

```

```

        reg_q <= {reg_q[30:0], ~sub_add[32]};
        count <= count + 5'b1;
        if (count == 5'd31)
            busy <= 1'b0;
    end
end
end
endmodule

```

有符号除法器 DIV 用于实现 32 位有符号数除法运算，输入被除数 dividend、除数 divisor，在 start 有效后开始运算，输出 32 位商 q、32 位余数 r，并用 busy 表示当前处于运算状态。该模块在无符号除法运算流程基础上增加符号处理：先对输入操作数取绝对值完成除法运算，再根据被除数和除数的符号确定商的符号，同时保证余数符号与被除数符号相同。

四、测试模块建模

(1) 无符号除法器

(要求列写各建模模块的 test bench 模块代码)

```

module DIVU_tb();
    reg [31:0] dividend;
    reg [31:0] divisor;
    reg start;
    reg clock;
    reg reset;
    wire [31:0] q;
    wire [31:0] r;
    wire busy;

    reg [31:0] dividend_set [0:4];
    reg [31:0] divisor_set [0:3];
    integer i;
    integer j;

    DIVU uut(
        .dividend(dividend),
        .divisor(divisor),
        .start(start),
        .clock(clock),
        .reset(reset),
        .q(q),
        .r(r),

```

```

        .busy(busy)
    );

    always #1 clock = ~clock;

    initial begin
        clock = 0;
        reset = 1;
        start = 0;
        dividend = 0;
        divisor = 0;

        dividend_set[0] = 32'h00000000;
        dividend_set[1] = 32'hffffffff;
        dividend_set[2] = 32'haaaaaaaa;
        dividend_set[3] = 32'h55555555;
        dividend_set[4] = 32'h7fffffff;

        divisor_set[0] = 32'hffffffff;
        divisor_set[1] = 32'haaaaaaaa;
        divisor_set[2] = 32'h55555555;
        divisor_set[3] = 32'h7fffffff;

        #25 reset = 0;

        for (i = 0; i < 5; i = i + 1) begin
            for (j = 0; j < 4; j = j + 1) begin
                dividend = dividend_set[i];
                divisor = divisor_set[j];
                start = 1;
                #20 start = 0;
                wait(busy == 0);
                #20;
            end
        end

        #20;
        $finish;
    end

endmodule

```

(2) 有符号除法器

```

module DIV_tb();
    reg [31:0] dividend;
    reg [31:0] divisor;
    reg start;
    reg clock;
    reg reset;
    wire [31:0] q;
    wire [31:0] r;
    wire busy;

    reg [31:0] dividend_set [0:4];
    reg [31:0] divisor_set [0:3];
    integer i;
    integer j;

    DIV uut(
        .dividend(dividend),
        .divisor(divisor),
        .start(start),
        .clock(clock),
        .reset(reset),
        .q(q),
        .r(r),
        .busy(busy)
    );

    always #1 clock = ~clock;

    initial begin
        clock = 0;
        reset = 1;
        start = 0;
        dividend = 0;
        divisor = 0;

        dividend_set[0] = 32'h00000000;
        dividend_set[1] = 32'hffffffff;
        dividend_set[2] = 32'haaaaaaaa;
        dividend_set[3] = 32'h55555555;
        dividend_set[4] = 32'h7fffffff;

        divisor_set[0] = 32'hffffffff;
        divisor_set[1] = 32'haaaaaaaa;
        divisor_set[2] = 32'h55555555;

```

```

divisor_set[3] = 32'h7fffffff;

#5 reset = 0;

for (i = 0; i < 5; i = i + 1) begin
    for (j = 0; j < 4; j = j + 1) begin
        dividend = dividend_set[i];
        divisor = divisor_set[j];
        start = 1;
        #20 start = 0;
        wait(busy == 0);
        #20;
    end
end

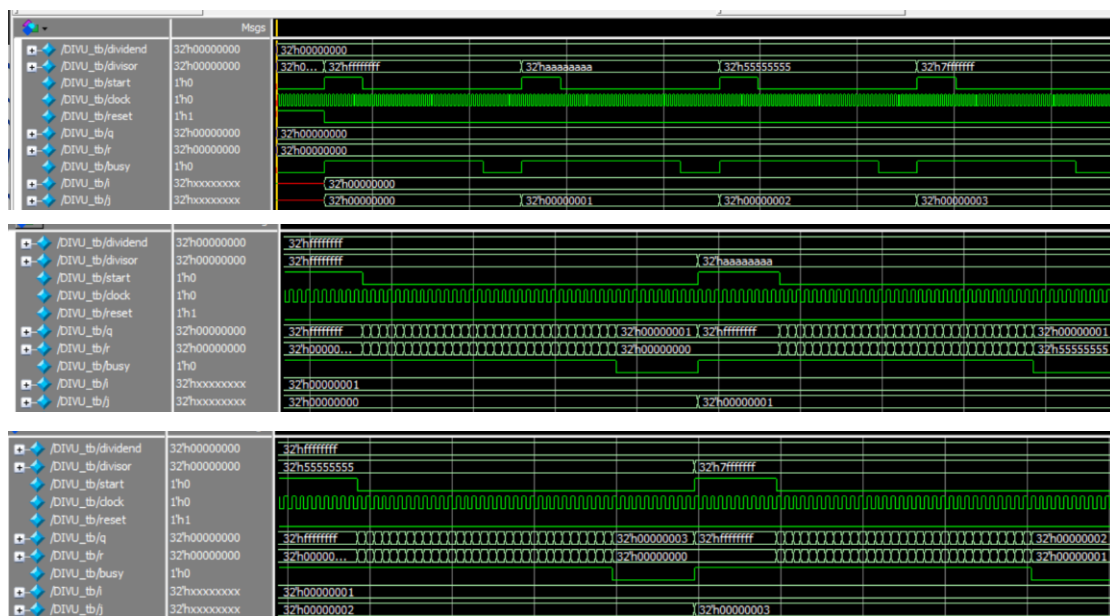
#5;
$finish;
end

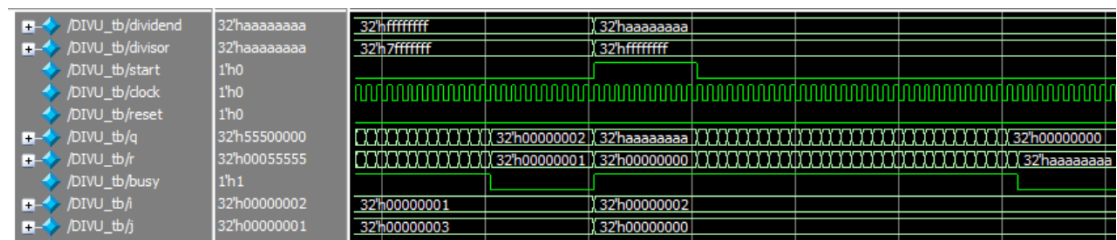
endmodule

```

五、实验结果

(1) 无符号除法器





(2) 有符号除法器

